

Lesson 6 — Capstone: Brick Breaker

One lesson: 2 hours 30 minutes · keep this sheet

Name _____

Class _____

Date _____

Cheat sheet

A class

```
class Brick:
    def __init__(self, x, y, w, h, hits):
        self.x = x
        self.y = y
        self.hits = hits
        self.alive = True

    def colour(self):
        return "orange" if self.hits >= 2
    else "blue"

    def box(self):
        return (self.x, self.y, self.w,
                self.h)

brick = Brick(40, 60, 68, 24, hits=2)
print(brick.x)          # 40
print(brick.colour()) # orange
```

self is not magic. `brick.colour()` calls `Brick.colour(brick)` — `self` is that first argument.

Use a class when a thing has **data and behaviour** and there are **many of them**. A one-off bag of values is still better as a dictionary.

Levels as data

```
LEVELS = [[
    [1, 1, 1, 1],
    [1, 0, 0, 1],
]]
```

```
for row in range(len(layout)):
    for column in range(len(layout[row])):
        kind = layout[row][column] # ROW
        FIRST
        if kind == 0: continue
        x = MARGIN + column * (BRICK_W +
        GAP)
        y = TOP      + row      * (BRICK_H +
        GAP)
```

Which side was hit?

```
ox = min(b.right, k.right) - max(b.left,
k.left)
oy = min(b.bottom, k.bottom) - max(b.top,
k.top)

if oy < ox: speed_y = -speed_y
else:      speed_x = -speed_x
```

The **smaller** overlap is the side it came in through.

One brick per frame

```
break # after handling a hit
```

Otherwise a ball between two bricks flips twice and carries straight on.

Saving

```
SAVE = Path(__file__).parent /
"highscore.json"

def load():
    try:
        return json.loads(SAVE.read_text())
    except Exception:
        return 0

def save(v):
    try:
        SAVE.write_text(json.dumps({"high_score":
v}))
    except Exception:
        pass
```

The `try` blocks are **part of the feature**. A game that crashes because it cannot save is worse than one that quietly does not.

Steering

```
offset = (ball.x - centre) / (paddle.w / 2)
# -1 .. +1
```

Divide by the maximum to get a fraction. Health bars, fades, sliders — all the same move.

Section A — Recall

A1. What is `self`? Explain what happens when you write `brick.colour()`. [3 marks]

A2. When is a class a better choice than a dictionary? Name one thing in a game that should stay a dictionary. [3 marks]

A3. In `layout[row][column]`, why is the row index first? [2 marks]

A4. How do you work out which *side* of a brick the ball hit? Why does always flipping the vertical speed go wrong? [3 marks]

A5. Give three different reasons the save file might fail to load, and say what the game should do in each case. [3 marks]

Section B — Predict the output

B1. How many bricks does this build, and what shape? [3 marks]

```
[[1, 0, 1, 0],
 [0, 2, 0, 2],
 [1, 1, 1, 1]]
```

B2. What error does this give, and what does the message tell you about `self`? [4 marks]

```
class Brick:
    def colour():
        return "blue"

b = Brick()
print(b.colour())
```

B3. The ball is travelling right and slightly up, and clips the **left edge** of a brick. Which overlap is smaller, and which speed should flip? Describe what the player sees if the code always flips `speed_y`. [4 marks]

B4. Someone edits `highscore.json` to say `hello`. What happens when the game starts, and why? [3 marks]

Section C — Find and fix the bug

C1. Every `Brick` reports the same position, whichever one you ask. What is wrong? [4 marks]

```
class Brick:
    x = 0
    y = 0

    def __init__(self, x, y):
        x = x
        y = y
```

C2. Bricks disappear two or three at a time, and the ball often carries straight on afterwards instead of bouncing. What is missing? [3 marks]

C3. A destroyed brick stops being drawn but the ball still bounces off it. Which of the two records was cleaned up, and which was not? Write the fix. [4 marks]

Section D — Build it

This is the capstone. You have 85 minutes. Most of it you have written before.

- ☐ **Checkpoint 1.** A tkinter window with a paddle you steer and a ball that bounces off the left, right and top walls. Falling off the bottom loses a life. (Lessons 3–5.)
- ☐ **Checkpoint 2.** Write a `Brick` class with `__init__`, `box()` and `hit()`. Make one and draw it.
- ☐ **Checkpoint 3.** Write `LEVELS` as nested lists and `build_level()` to turn one into `Brick` objects. Change digits and reload to prove it works.
- ☐ **Checkpoint 4.** Make the ball destroy bricks, bouncing off the correct *side*. Add the `break`. Add a score.

- ☐ **Checkpoint 5.** Add lives, at least three levels, a game-over screen and a restart. Make tough bricks change colour after the first hit.
- ☐ **Checkpoint 6.** Save the high score to a file so it survives closing the game. Test it by deliberately corrupting the file.

HOW THIS IS MARKED

Not on features. On four things: does it run; are the levels separate from the game logic; can you explain any method I point at; and did you change something to make it yours?</p>
A rougher game with an original idea in it beats a perfect copy of the example.

Section E — Design challenge

NO RIGHT ANSWER

E1. Your save file is plain text that a player could open and edit. Is that a problem? When would it be? What would you do about someone setting their high score to 999999 — and is **"nothing"** an acceptable answer?

E2. Your layouts are 8 columns wide. What would have to change for a 20-wide level? If the answer is "nothing", your code is good. If it is "several numbers", list them and say why they are there.

E3. Design a level format that can also say "this brick is red", "this brick drops a power-up", "this brick cannot be broken". What does it look like? What did you give up?

E4. The big one. You have built four games in this track: a text adventure, a steerable player, Snake, and Brick Breaker. **What is the same in all four?** Design the leftover part — the thing that would be useful in a fifth game. What must always stay specific to one game?

Stretch goals

1. Power-ups dropped by bricks.
2. **A level editor** — click a grid, print the array, paste it back. About 30 lines.
3. Save the whole game, not just the score, so you can quit and resume.
4. Tough bricks that look tough.
5. **Make it yours.** Gravity. Moving bricks. Two players. This is the real one.